

called **PE (Portable Executable)**. As you can see, the BIOS under UEFI very much looks like a little operating system itself which understands partitions, file systems, executables, etc. It even has a shell with some standard commands.

The boot code still needs to pick one of the bootloader programs to load Linux or Windows, or whatever operating system, but there may be many partitions with operating systems and given so much choice, which one should it pick? This is decided by the UEFI boot manager, which you can think of as a boot menu with different entries and a configurable order in which to try the different boot options. Changing the menu and the default bootloader is very easy and can be done from within the currently executing operating system. As before, the bootloader will continue loading the operating system of choice.

This is by no means the full story. UEFI is very flexible and highly standardized, and contains many advanced features. However, this is enough for now. In Chap. 9, we will pick up UEFI again when we discuss an interesting feature known as **Secure Boot**, which allows a user to be sure that the operating system is booted as intended and with the correct software.

1.4 THE OPERATING SYSTEM ZOO

Operating systems have been around now for over half a century. During this time, quite a variety of them have been developed, not all of them widely known. In this section, we will briefly touch upon nine of them. We will come back to some of these different kinds of systems later in the book.

1.4.1 Mainframe Operating Systems

At the high end are the operating systems for mainframes, those room-sized computers still found in major corporate data centers. These computers differ from personal computers in terms of their I/O capacity. A mainframe with 1000 hard disks and many terabytes of data are not unusual; a personal computer with these specifications would be the envy of its friends. Mainframes are also making something of a comeback as high-end servers for large-scale electronic commerce sites, banking, airline reservations, and servers for business-to-business transactions.

The operating systems for mainframes are heavily oriented toward processing many jobs at once, most of which need prodigious amounts of I/O. They typically offer three kinds of services: batch, transaction processing, and timesharing. A batch system is one that processes routine jobs without any interactive user present. Claims processing in an insurance company or sales reporting for a chain of stores is typically done in batch mode. Transaction-processing systems handle large numbers of small requests, for example, check processing at a bank or airline reservations. Each unit of work is small, but the system must handle hundreds or thousands per second. Timesharing systems allow multiple remote users to run jobs on the computer at once, such as querying a big database. These functions are closely

related; mainframe operating systems often perform all of them. An example mainframe operating system is Z/OS, the successor of OS/390, which in turn was a direct descendant of OS/360. However, mainframe operating systems are gradually being replaced by UNIX variants such as Linux.

1.4.2 Server Operating Systems

One level down are the server operating systems. They run on servers, which are either very large personal computers, workstations, or even mainframes. They serve multiple users at once over a network and allow the users to share hardware and software resources. Servers can provide print service, file service, database service, or Web service. Internet providers run many server machines to support their customers and Websites use servers to store the Web pages and handle the incoming requests. Typical server operating systems are Linux, FreeBSD, Solaris, and the Windows Server family.

1.4.3 Personal Computer Operating Systems

The next category is the personal computer operating system. Modern ones all support multiprocessing, often with dozens of programs started up at boot time, and multiprocessor architectures. Their job is to provide good support to a single user. They are widely used for word processing, spreadsheets, games, and Internet access. Common examples are Windows 11, macOS, Linux, and FreeBSD. Personal computer operating systems are so widely known that probably little introduction is needed. In fact, many people are not even aware that other kinds exist.

1.4.4 Smartphone and Handheld Computer Operating Systems

Continuing on down to smaller and smaller systems, we come to tablets (like Apple's iPad), smartphones and other handheld computers. A handheld computer, originally known as a **PDA (Personal Digital Assistant)**, is a small computer that can be held in your hand during operation. Smartphones and tablets are the best-known examples. As we have already seen, this market is currently dominated by Google's Android and Apple's iOS. Most of these devices boast multicore CPUs, GPS, cameras and other sensors, copious amounts of memory, and sophisticated operating systems. Moreover, all of them have more third-party applications (**apps**) than you can shake a (USB) stick at. Google has over 3 million Android apps in the Play Store and Apple has over 2 million in the App Store.

1.4.5 The Internet of Things and Embedded Operating Systems

The **IOT (Internet of Things)** comprises all the billions of physical objects with sensors and actuators that are increasingly connected to the network, such as fridges, thermostats, security camera's motion sensors, and so on. All of these

devices contain small computers and most of them run small operating systems. In addition, we may have even more embedded systems controlling devices that are not connected to a network at all. Examples include traditional microwave ovens and washing machines. Such systems do not accept user-installed software, so the main property which distinguishes such embedded systems from the computers we discussed earlier is the certainty that no untrusted software will ever run on it. Few microwave ovens allow you to download and run new applications—all the software is in ROM. However, even that is changing. Some high-end cameras have their own app stores that allow users to install custom apps for in-camera editing, multiple exposures, different focus algorithms, different image compression algorithms, and more.

Nevertheless, for most embedded systems there is no need for protection between applications, leading to design simplification. Conversely, such operating systems may provide more support for functionality such as real-time scheduling or low-power networking which are important in many embedded systems. Systems such as Embedded Linux, QNX, and VxWorks are popular in this domain. For severely resource-constrained devices, the operating system should be able to run in just a few kilobytes. For instance, RIOT, an open source operating system for IoT devices, can run in less 10 KB and support systems that range from 8-bit microcontrollers to general-purpose 32-bit CPUs. The TinyOS operating similarly offers a very small footprint, making it popular in sensor nodes.

1.4.6 Real-Time Operating Systems

Real-time systems are characterized by having time as a key parameter. For example, in industrial process-control systems, real-time computers have to collect data about the production process and use it to control machines in the factory. Often there are hard deadlines that must be met. For example, if a car is moving down an assembly line, certain actions must take place at certain instants of time. If, for example, a welding robot welds too early or too late, the car will be ruined. If the action absolutely *must* occur at a certain moment (or within a certain range), we have a **hard real-time system**. Many of these are found in industrial process control, avionics, military, and similar application areas. These systems must provide absolute guarantees that a certain action will occur by a certain time.

A **soft real-time system** is one where missing an occasional deadline, while not desirable, is acceptable and does not cause any permanent damage. Digital audio or multimedia systems fall in this category. Smartphones are also soft real-time systems.

Since meeting deadlines is crucial in (hard) real-time systems, sometimes the operating system is simply a library linked in with the application programs, with everything tightly coupled and no protection between parts of the system. An example of this type of real-time system is eCos.

We should emphasize that the categories of IoT, embedded, real-time and even handheld systems overlap considerably. Many of them have at least some soft real-time aspects. The embedded and real-time systems run only software put in by the system designers; users cannot add their own software, which makes protection easier.

1.4.7 Smart Card Operating Systems

The smallest operating systems run on smart cards, which are credit-card-sized devices containing a CPU. They have very severe processing power and memory constraints. Some are powered by contacts in the reader into which they are inserted, while contactless smart cards are inductively powered (which greatly limits what they can do.) Some of them can handle only a single function, such as electronic payments, but others can handle multiple functions. Often these are proprietary systems.

Some smart cards are Java oriented. This means that the ROM on the smart card holds an interpreter for the Java Virtual Machine (JVM). Java applets (small programs) are downloaded to the card and are interpreted by the JVM interpreter. Some of these cards can handle multiple Java applets at the same time, leading to multiprogramming and the need to schedule them. Resource management and protection also become an issue when two or more applets are present at the same time. These issues must be handled by the (usually extremely primitive) operating system present on the card.

1.5 OPERATING SYSTEM CONCEPTS

Most operating systems provide certain basic concepts and abstractions such as processes, address spaces, and files that are central to understanding them. In the following sections, we will look at some of these basic concepts ever so briefly, as an introduction. We will come back to each of them in great detail later in this book. To illustrate these concepts we will, from time to time, use examples, generally drawn from UNIX. Similar examples typically exist in other systems as well, however, and we will study some of them later.

1.5.1 Processes

A key concept in all operating systems is the **process**. A process is basically a program in execution. Associated with each process is its **address space**, a list of memory locations from 0 to some maximum, which the process can read and write. The address space contains the executable program, the program's data, and its stack. Also associated with each process is a set of resources, commonly including registers (including the program counter and stack pointer), a list of open files,